

# SYNTHETIC OCEANS

## Relatório de Projeto

Unidade Curricular: Programação Orientada a Objetos

Licenciatura em Engenharia Informática

Instituto Politécnico de Setúbal

Trabalho realizado por:

Daniel Alexandre Silveiro Grancho

Daniel Amaro Rosado

Ano Letivo 2025/2026

Junho 2026

# 1. Apresentação do Projeto

Synthetic Oceans é um jogo de arcade inspirado no clássico Space Invaders, desenvolvido em Java com interface gráfica em JavaFX.

O tema é subaquático/sci-fi: o jogador controla um submarino neon na base do ecrã e deve defender-se de vagas de criaturas abissais bioluminescentes que avançam em formação.

## 1.1 Objetivo do Jogo

O objetivo é sobreviver o maior número de vagas possível e obter a pontuação mais alta, eliminando os inimigos antes que estes alcancem a base do ecrã ou destruam o jogador com os seus projéteis.

## 1.2 Regras Principais

- O jogador move-se horizontalmente na base do ecrã (teclas ← / → ou A / D).
- Só é permitido um projétil do jogador ativo em simultâneo (tecla ESPAÇO).
- Os inimigos movem-se em bloco para os lados; ao atingirem a borda lateral descem uma linha e invertem a direção.
- Existem 3 barricadas destrutíveis que absorvem projéteis de ambos os lados.
- Um invasor aleatório (bónus) atravessa o topo do ecrã periodicamente, valendo entre 50 e 300 pontos.
- O jogo organiza-se em vagas, com dificuldade crescente a cada vaga superada.

## 1.3 Condições de Derrota

- O jogador perde todas as 3 vidas por colisão com projéteis inimigos.
- A frota inimiga atinge a linha base onde o jogador se encontra (derrota imediata, independentemente das vidas restantes).

## 1.4 Interação com o Utilizador

| Tecla   | Ação                    |
|---------|-------------------------|
| ← / A   | Mover para a esquerda   |
| → / D   | Mover para a direita    |
| ESPAÇO  | Disparar projétil       |
| ESC / P | Pausar / Retomar o jogo |

---

## 2. Modelação do Domínio

O domínio do jogo é composto por seis categorias principais de entidades, todas derivadas de uma superclasse abstrata comum — `EntidadeJogo` — que encapsula as propriedades espaciais (posição, dimensões) e a deteção de colisões.

### 2.1 Entidades Identificadas

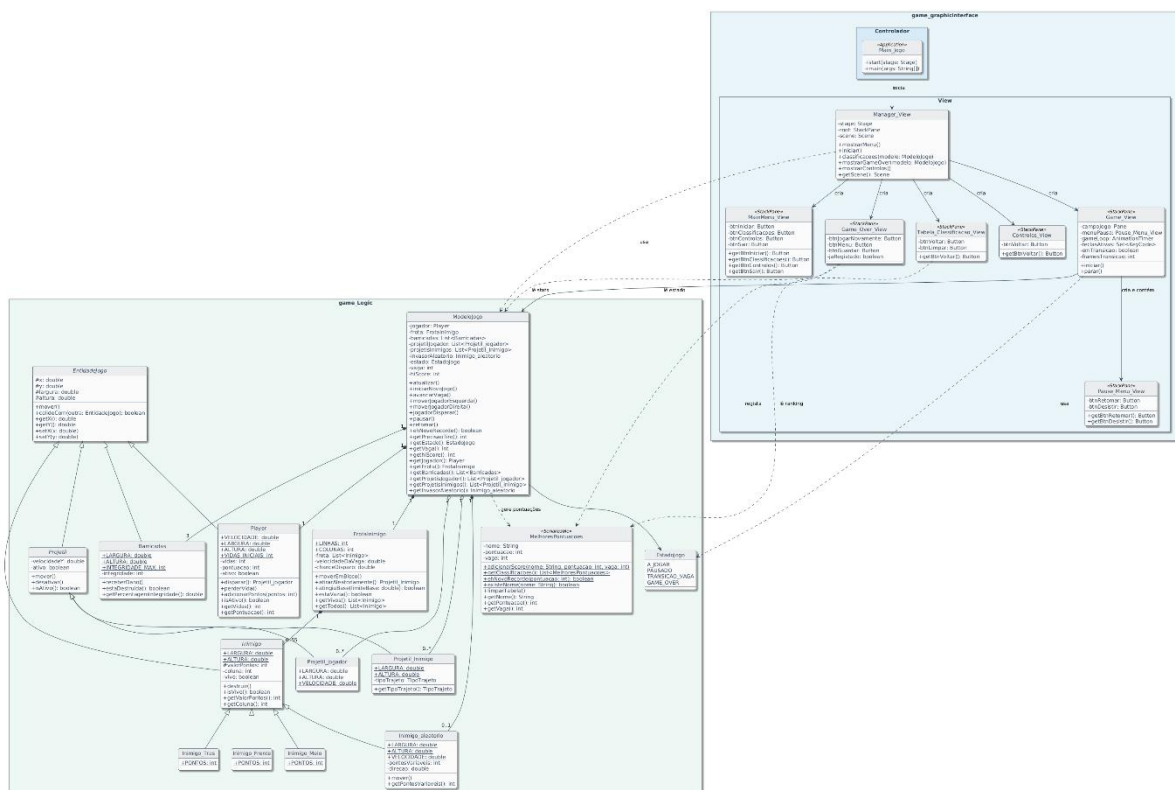
| Entidade                         | Responsabilidade                                                                                                          |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>EntidadeJogo</code>        | Superclasse abstrata com posição (x, y), dimensões e deteção de colisões (AABB). Define o contrato <code>mover()</code> . |
| <code>Player</code>              | Representa o submarino controlado pelo utilizador. Gere vidas, pontuação, movimento horizontal e criação de projéteis.    |
| <code>Inimigo (abstrata)</code>  | Superclasse dos invasores da frota. Guarda o índice de coluna original, o valor em pontos e o estado vivo/destruído.      |
| <code>Inimigo_Frente</code>      | Inimigos das filas 3-4 da grelha. Valem 10 pontos.                                                                        |
| <code>Inimigo_Meio</code>        | Inimigos das filas 1-2 da grelha. Valem 20 pontos.                                                                        |
| <code>Inimigo_Tras</code>        | Inimigos da fila 0 da grelha. Valem 30 pontos.                                                                            |
| <code>Inimigo_aleatorio</code>   | Invasor bónus que atravessa o topo horizontalmente. Não ataca; vale 50-300 pontos (aleatório).                            |
| <code>FrotaInimigo</code>        | Gere a grelha 5x11 de inimigos, o movimento em bloco, os disparos e a velocidade crescente por vaga.                      |
| <code>Projetil (abstrata)</code> | Superclasse dos projéteis. Define velocidade vertical, estado ativo/inativo e o método <code>mover()</code> .             |
| <code>Projetil_jogador</code>    | Laser retilíneo do jogador, velocidade de -9.0 px/frame (sobe).                                                           |
| <code>Projetil_Inimigo</code>    | Projétil inimigo com dois tipos de trajetória: RETO (lento) ou ZIGZAG.                                                    |
| <code>Barricadas</code>          | Estrutura defensiva estática com 10 pontos de integridade. Degrada-se com cada impacto.                                   |
| <code>ModeloJogo</code>          | Controlador central da lógica: coordena entidades, colisões, estado do jogo e progressão de vagas.                        |
| <code>EstadoJogo</code>          | Enumeração com os estados: MENU, A_JOGAR, PAUSADO, TRANSICAO_VAGA, GAME_OVER.                                             |
| <code>MelhoresPontuacoes</code>  | Gestão da tabela de classificações (top 10), com persistência em ficheiro binário (Serializable).                         |

## 2.2 Relações Entre Entidades

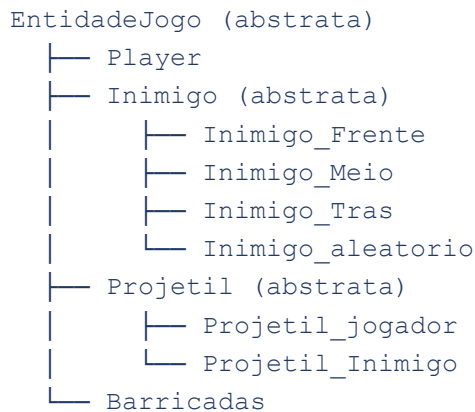
- ModeloJogo agrega (composição) Player, FrotalInimigo, Barricadas, listas de Projtil e Inimigo\_aleatorio.
- FrotalInimigo contém uma lista de Inimigo (polimorfismo: Inimigo\_Frente, Inimigo\_Meio, Inimigo\_Tras).
- Player cria instâncias de Projtil\_jogador; FrotalInimigo cria instâncias de Projtil\_Inimigo.
- Inimigo\_aleatorio herda de Inimigo (com índice de coluna sentinela -1).
- Projtil\_jogador e Projtil\_Inimigo herdam de Projtil, que herda de EntidadeJogo.
- MelhoresPontuacoes é usada por ModeloJogo para registrar e consultar os high scores.

### 3. Diagrama de Classes UML

O diagrama seguinte apresenta a estrutura hierárquica do modelo de domínio. A hierarquia principal parte de EntidadeJogo, da qual derivam as classes Player, Inimigo (abstrata), Projétil (abstrata) e Barricadas.



## Hierarquia principal:



Classes de suporte: ModeloJogo, FrotalInimigo, EstadoJogo (enum), MelhoresPontuacoes.

---

## 4. Arquitetura da Aplicação

A aplicação segue uma arquitetura MVC (Model-View-Controller) com separação clara entre camadas, organizada em dois pacotes principais:

- game\_Logic — modelo de domínio (lógica de jogo, entidades, estado)
- game\_graphicInterface — interface gráfica (View) e ponto de entrada (Controlador)

### 4.1 Modelo de Domínio (game\_Logic)

O modelo é completamente independente de JavaFX. Contém todas as classes de entidade, o ModeloJogo que orquestra o ciclo de jogo e a persistência de pontuações. O ModeloJogo expõe apenas getters para a View, garantindo encapsulamento total da lógica.

### 4.2 Controlo da Aplicação

O controlo é partilhado entre Game\_View e Manager\_View:

- Manager\_View: gere a navegação entre ecrãs (menu, jogo, game over, classificações, controlos). Cria o ModeloJogo e instancia as Views correspondentes.
- Game\_View: contém o AnimationTimer (game loop a 60 fps), processa o input do teclado e chama modelo.atualizar() a cada frame, atualizando depois as formas JavaFX conforme o estado do modelo.

### 4.3 Interface Gráfica (JavaFX)

| Classe View               | Função                                                                                                |
|---------------------------|-------------------------------------------------------------------------------------------------------|
| Main_Jogo                 | Ponto de entrada JavaFX; cria o Stage e a Scene principal.                                            |
| Manager_View              | Gestor de navegação; troca o conteúdo do StackPane raiz.                                              |
| MainMenu_View             | Menu principal com animação de submarinos neon em SVGPath.                                            |
| Game_View                 | Vista de jogo; usa Pane com formas JavaFX (Rectangle, Ellipse, Polygon) re-renderizadas a cada frame. |
| Pause_Menu_View           | Overlay semitransparente sobreposto ao jogo quando pausado.                                           |
| Game_Over_View            | Ecrã de fim de jogo com estatísticas, inserção de iniciais e botões de navegação.                     |
| Tabela_Classificacao_View | Tabela top-10 de classificações com opção de limpar.                                                  |
| Controlos_View            | Ecrã informativo com os controlos do jogo.                                                            |

A View nunca acede diretamente ao estado interno das entidades para o modificar — apenas lê dados via getters do ModeloJogo.

## 5. Funcionalidades Implementadas

### 5.1 Jogo

- Grelha de inimigos 5×11 com três tipos (Frente 10pts, Meio 20pts, Trás 30pts).
- Movimento em bloco dos inimigos: ao tocar na borda lateral, a frota desce e inverte direção.
- Velocidade da frota fixa por vaga e crescente a cada nova vaga (base 0.2 + 0.3×vaga).
- Disparo de projéteis inimigos com dois tipos de trajetória: RETO e ZIGZAG.
- Invasor aleatório bónus que atravessa o topo com pontuação variável (50–300 pts).
- 3 barricadas destrutíveis com 10 pontos de integridade, com efeito visual de degradação (rachas).
- Detecção de colisões AABB entre todas as combinações de entidades.
- Sistema de vidas (3) e pontuação cumulativa por vaga.

- Progressão de vagas com transição visual (overlay com estatísticas de precisão).
- Condições de derrota: sem vidas ou frota atinge a base.

## 5.2 Interface e Navegação

- Menu principal com submarinos neon animados (SVGPath JavaFX).
- Implementação de música de fundo em loop contínuo utilizando a API de Media do JavaFX (MediaPlayer), configurada para arrancar com o jogo.
- HUD em jogo: SCORE, HI-SCORE, VAGA, VIDAS.
- Ecrã de pausa (ESC / P) com overlay semitransparente.
- Ecrã de Game Over com pontuação, vaga alcançada, precisão de tiro e registo de iniciais.
- Tabela de classificações top-10 com persistência entre sessões (ficheiro .dat).
- Ecrã de controlos descritivo.
- Paleta visual neon coerente: jogador ciano (#00f5ff), inimigos por camada (ciano / roxo / laranja), fundo gradiente azul-profundo.

## 5.3 Persistência

- MelhoresPontuacoes implementa Serializable; as classificações são guardadas em pontuacoes.dat e carregadas automaticamente no arranque.
- Proteção contra registo duplicado no ecrã de Game Over (flag jaRegistado).
- Geração automática de nome (User N) se o jogador não inserir iniciais.

---

# 6. Aplicação dos Conceitos de Programação Orientada a Objetos

## 6.1 Encapsulamento

Todos os campos das entidades são `private` ou `protected`, acessíveis apenas através de getters e setters com lógica de validação quando necessário. Por exemplo, `Player.adicionarPontos()` verifica que os pontos não são negativos antes de os adicionar. O `ModeloJogo` não expõe os campos internos diretamente — apenas fornece referências não modificáveis (`Collections.unmodifiableList`) às listas que a View necessita de ler.

## 6.2 Herança

Existem duas hierarquias de herança bem definidas:

- EntidadeJogo → Inimigo → Inimigo\_Frente / Inimigo\_Meio / Inimigo\_Tras / Inimigo\_aleatorio: partilha de posição, dimensões, colisão e estado vivo/destruído.
- EntidadeJogo → Projétil → Projétil\_jogador / Projétil\_Inimigo: partilha de movimento vertical, estado ativo e método desativar().
- EntidadeJogo → Barricadas / Player: reutilização do AABB e dos campos de posição.

O Inimigo\_aleatorio herda de Inimigo apesar de não pertencer à grelha — o índice de coluna é definido como -1 (valor sentinela), evitando duplicação de lógica de estado de vida.

### 6.3 Polimorfismo

O polimorfismo é utilizado de forma sistemática:

- FrotalInimigo mantém uma List<Inimigo> e itera sobre ela invocando métodos da superclasse, independentemente do tipo concreto (Frente, Meio, Trás).
- ModeloJogo processa as listas de Projétil\_jogador e Projétil\_Inimigo através do método mover() herdado de Projétil, e chama desativar() / isAtivo() sem conhecer o tipo concreto.
- Projétil\_Inimigo sobrepõe (override) moverVerticalmente() para adicionar o comportamento de zigzag quando TipoTrajeto.ZIGZAG, enquanto o trajetório reto usa a implementação base.
- O método abstrato mover() em EntidadeJogo obriga todas as subclasses a definir o seu comportamento de movimento.

## 7. Tratamento de Exceções

As seguintes situações de erro foram identificadas e tratadas:

### 7.1 Validação de Dados

Em Player.adicionarPontos(int pontos), é lançada uma IllegalArgumentException se o valor recebido for negativo, protegendo a integridade da pontuação:

```
if (pontos < 0) throw new IllegalArgumentException("Pontos nao podem ser negativos.");
```

### 7.2 Persistência de Pontuações

Em MelhoresPontuacoes, as operações de leitura e escrita do ficheiro binário são protegidas com try-catch para IOException e ClassNotFoundException. Em caso de erro, a aplicação continua com uma lista vazia, evitando falhas em cascata:

```
} catch (IOException | ClassNotFoundException e) {
    return new ArrayList<>();
}
```

### 7.3 Robustez de Nomes



O construtor de `MelhoresPontuacoes` trunca automaticamente nomes com mais de 10 caracteres (substring), evitando estouro de campo na tabela de classificações.

## 7.4 Proteção contra Registo Duplicado

A flag `jaRegistado` em `Game_Over_View` garante que o registo de pontuação é chamado no máximo uma vez, mesmo que o utilizador clique em múltiplos botões rapidamente.

## 7.5 Carregamento de Recursos Multimédia

Na inicialização do áudio no método `Manager_View.iniciarMusicaFundo()`, o bloco try-catch previne que a aplicação termine abruptamente caso o ficheiro de música .mp3 falhe a leitura ou o URL retorne null. Adicionalmente, verifica-se a integridade do caminho garantindo que o `MediaPlayer` apenas é executado de forma segura.

---

# 8. Testes Unitários

Os testes unitários cobrem o modelo de domínio (pacote `game_Logic`), sem qualquer dependência de `JavaFX`.

Os principais cenários testados são descritos abaixo. Em todas as classes, testámos também os construtores para verificar se a instanciação e o encapsulamento foram garantidos com sucesso:

## 8.1 PlayerTest

| Método Testado                 | Cenário                                                                                    |
|--------------------------------|--------------------------------------------------------------------------------------------|
| <code>adicionarPontos()</code> | Adicionar pontos positivos: verifica incremento correto.                                   |
| <code>adicionarPontos()</code> | Adicionar pontos negativos: verifica lançamento de <code>IllegalArgumentException</code> . |
| <code>perderVida()</code>      | Perder vidas até zero: verifica que <code>isAtivo()</code> passa a false.                  |
| <code>disparar()</code>        | Criação de projétil: verifica que o projétil é criado centrado no jogador.                 |

## 8.2 BarricadasTest

| Método Testado              | Cenário                                                          |
|-----------------------------|------------------------------------------------------------------|
| receberDano()               | 10 impactos seguidos: verifica que estaDestruida() retorna true. |
| getPercentagemIntegridade() | Após 5 impactos: verifica que o valor é 0.5.                     |
| receberDano()               | Dano abaixo de zero: verifica que integridade não fica negativa. |

## 8.3 FrotaInimigoTest

| Método Testado         | Cenário                                                |
|------------------------|--------------------------------------------------------|
| estaVazia()            | Frota nova: verifica que não está vazia.               |
| getVivos()             | Destruir todos: verifica que estaVazia() retorna true. |
| atingiuBase()          | Y dos inimigos acima da base: verifica retorno false.  |
| atirarAleatoriamente() | Frota vazia: verifica retorno null.                    |

## 8.4 MelhoresPontuacoesTest

| Método Testado   | Cenário                                                                   |
|------------------|---------------------------------------------------------------------------|
| adicionarScore() | Inserção de pontuação: verifica ordenação decrescente.                    |
| adicionarScore() | Tabela completa (10 entradas): verifica que pontuação inferior não entra. |
| ehNovoRecorde()  | Pontuação superior ao primeiro lugar: verifica true.                      |
| limparTabela()   | Após limpar: verifica que lista fica vazia.                               |

---

## 8.5 ModeloJogoTest

| Método Testado                          | Cenário                                                                                                                    |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| testMovimentoDoJogadorRespeitaLimites() | Forçar movimento contínuo para os lados: verifica que a coordenada X não ultrapassa os limites do ecrã.                    |
| jogadorDisparar()                       | Tentar disparar múltiplas vezes seguidas: verifica que a lista de projéteis mantém apenas 1 tiro ativo em simultâneo.      |
| pausar() / retomar()                    | Alternância de estados: verifica a transição correta na enumeração EstadoJogo entre A_JOGAR e PAUSADO.                     |
| avancarVaga()                           | Transição de vaga: verifica o incremento da variável vaga, reinício do contador de tiros e limpeza de listas de projéteis. |

## 8.6 EntidadeJogoTest (Lógica de Colisão)

| Método Testado | Cenário                                                                                                          |
|----------------|------------------------------------------------------------------------------------------------------------------|
| colideCom()    | Sobreposição total e parcial: verifica se a deteção AABB devolve true quando os hitboxes se cruzam.              |
| colideCom()    | Entidades separadas: verifica que a deteção AABB devolve false quando não há contacto, mesmo tocando nas bordas. |

## 8.7 Inimigo\_aleatorioTest

| Método Testado               | Cenário                                                                                                                                            |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| testSorteioDePontosValidos() | Instanciação do OVNI: verifica se os pontos atribuídos pertencem estritamente ao array de valores permitidos (50 a 300).                           |
| mover()                      | Movimento horizontal por direção: verifica se soma ou subtrai corretamente à coordenada X dependendo se a direção é +1 (direita) ou -1 (esquerda). |

## 8.8 Projétil\_InimigoTest

| Método Testado       | Cenário                                                                                                                                   |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| resolverVelocidade() | Instanciação por tipo: verifica se o projétil RETO sofre a redução para 60% da velocidade base, e se o ZIGZAG mantém a velocidade total.  |
| moverVerticalmente() | Trajetória ZIGZAG: simulação de frame; verifica a oscilação na coordenada X resultante do cálculo trigonométrico ( $\text{Math.sin}()$ ). |

## 8.9 Classes de Suporte e Configuração

| Método Testado                | Cenário                                                                                                                            |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>InimigoTest</b> (Abstrata) | Verifica atribuição do estado vivo = true, constantes de dimensão e transição de estado no método destruir().                      |
| <b>Projétil_jogadorTest</b>   | Valida a correta passagem de constantes dimensionais e garante que a velocidade é negativa (-9.0) para assegurar a subida no ecrã. |

|                           |                                                                                               |
|---------------------------|-----------------------------------------------------------------------------------------------|
| <b>Inimigo_FrenteTest</b> | Validação de construtor: garante a atribuição do valor de 10 pontos repassados à superclasse. |
|---------------------------|-----------------------------------------------------------------------------------------------|

---

## 9. Dificuldades Encontradas

### 9.1 Controlo dos inimigos

Foi desafiador conseguir utilizar o sistema AABB e controlar o ritmo de disparos para o jogo conseguir ser jogável para quem não tem não sabe jogar e conseguir apenas passar pelo menos um nível e ir progredindo de forma continua.

### 9.3 Velocidade Variável vs. Fixo por Vaga

A implementação inicial acelerava os inimigos progressivamente conforme iam sendo destruídos (como no Space Invaders original). Esta abordagem tornou o balanceamento difícil e o comportamento imprevisível. A decisão final foi definir a velocidade como fixa para toda a vaga (calculada no construtor de `FrotaInimigo`), aumentando apenas entre vagas, o que resultou numa dificuldade mais previsível e justa.

### 9.4 Registo Duplicado de Pontuação

O ecrã de Game Over tem vários botões que podem desencadear o registo de pontuação. Para evitar entradas duplicadas na tabela, foi introduzida a flag booleana `jaRegistado` que garante que `MelhoresPontuacoes.adicionarScore()` é chamado no máximo uma vez por sessão de Game Over.

---

## 10. Conclusão

O projeto Synthetic Oceans permitiu aplicar de forma integrada e prática os principais conceitos da Programação Orientada a Objetos num contexto de desenvolvimento de jogo completo.

A hierarquia de herança centrada em `EntidadeJogo` revelou-se uma base sólida que reduziu significativamente a duplicação de código. O polimorfismo simplificou o tratamento uniforme de inimigos e projéteis pelo `ModeloJogo`. O encapsulamento garantiu que a View nunca corrompeu o estado do modelo, e a separação MVC facilitou a manutenção e a testabilidade independente das camadas.

Os conceitos mais desafiantes foram a gestão eficiente da renderização frame a frame com formas JavaFX (sem Canvas), a definição da estratégia de disparo por coluna e o balanceamento da dificuldade progressiva. As soluções adotadas — marcação de nós com `userData`, índice de coluna imutável e velocidade fixa por vaga — demonstraram a importância de decisões de design tomadas cedo no desenvolvimento.

O resultado final é um jogo funcional, visualmente coerente com a estética neon/subaquática definida na Fase 1, com persistência de classificações, sistema de vagas progressivo e uma interface navegável completa.